

Tập 24 - BPF Maps

BPF Maps: Hash, LRU, Array, Per-CPU — Vũ khí hiệu năng của Cilium

Phần 3 — Cilium · `#ebpf` `#bpfmaps` `#kernel` `#hashmap` `#performance`



Mục tiêu tập này

- BPF Maps là gì — cầu nối giữa kernel space và user space
- 4 loại Map quan trọng nhất trong Cilium
- Tại sao BPF Maps thay thế được iptables chains
- Inspect BPF Maps thực tế để hiểu Cilium đang "nghĩ" gì

Prerequisites: Cilium đang chạy trên cluster (từ Tập 23)

BPF Maps là gì?

BPF Maps = Shared memory giữa:



Ví dụ thực tế:

- cilium-agent ghi policy vào BPF Map
- BPF program trong kernel đọc Map per-packet
- Quyết định ALLOW/DROP trong nanoseconds

Không cần syscall! Không cần context switch!

→ Đây là bí mật của Cilium performance

Tại sao eBPF cần BPF Maps?

Do cơ chế an toàn của Linux Kernel, eBPF program bị giới hạn rất ngặt nghèo:

- **Stateless (Không lưu trạng thái):** Mỗi khi một packet đi qua, eBPF program được gọi, chạy xong là kết thúc. Nó không thể tự lưu biến toàn cục để nhớ packet trước đó.
- **Không truy cập trực tiếp bộ nhớ User Space:** Kernel space và User space tách biệt. Không thể dùng con trỏ (pointer) để đọc/ghi chép chung.

☞ **BPF Maps giải quyết cả hai vấn đề:**

1. Là **Stateful Storage** giúp eBPF program lưu trữ trạng thái (ví dụ: conntrack, metrics).
2. Là kênh **IPC (Inter-Process Communication)** để Kernel và User space trao đổi thông tin cấu hình và dữ liệu cực nhanh.

Cơ chế hoạt động & Vòng đời của BPF Maps

1. Khởi tạo (Cilium-Agent hoặc Loader tạo Map trong Kernel qua syscall bpf())



2. Sử dụng (Helper functions trong Kernel Program / CLI ở User Space)

- bpf_map_lookup_elem(&map, &key)
- bpf_map_update_elem(&map, &key, &value, flags)
- bpf_map_delete_elem(&map, &key)



3. Lưu giữ (Pinned vào Virtual Filesystem: /sys/fs/bpf/)

- Map KHÔNG mất đi khi eBPF program dừng hoặc Cilium Agent restart!
- Đảm bảo data / policy không bị gián đoạn (zero-downtime).

4 loại Map quan trọng

Type	Use case	Đặc điểm
BPF_MAP_TYPE_HASH	Policy lookup: IP → rule	O(1) lookup, collision resistant
BPF_MAP_TYPE_LRU_HASH	Conntrack: flow state	Auto-evict oldest entry, fixed size
BPF_MAP_TYPE_ARRAY	Config, metrics counters	Index-based, always allocated
BPF_MAP_TYPE_PERCPU_HASH	Per-CPU packet counters	No lock contention, sum when read

Hash Map: Policy Lookup O(1)

`cilium_policy_<endpoint_id> (BPF_MAP_TYPE_HASH)`

Key: {src_ip, dst_ip, dst_port, protocol}

Value: {verdict: ALLOW/DROP, action_flags}

Lookup: O(1) – một hash function, một memory read
vs iptables: traverse n rules linearly

Khi packet đến:

1. BPF program extract 5-tuple từ packet header
2. `bpf_map_lookup_elem(&cilium_policy, &key)`
3. Return ALLOW → forward
Return NULL → DROP (default deny)

Với 100,000 policies: vẫn O(1)!

LRU Hash Map: Conntrack không cần lock

`cilium_ct_tcp4 (BPF_MAP_TYPE_LRU_HASH)`

Key: {src_ip, src_port, dst_ip, dst_port, proto}

Value: {state, last_seen, flags, rev_nat_index}

LRU = Least Recently Used eviction:

- Max entries: 512K connections (default)
- Khi full: evict connection lâu nhất không dùng
- Không block! Không crash!

vs conntrack kernel (nf_conntrack):

- nf_conntrack: spinlock on update
 - contention khi nhiều CPU
- BPF LRU: lockless per-CPU design
 - scale tuyến tính với số CPU cores

Per-CPU Hash Map: Counters lock-free

`cilium_metrics (BPF_MAP_TYPE_PERCPU_HASH)`

Mỗi CPU core có bản copy riêng của counter!

- Không cần atomic operation
- Không cần lock
- Tốc độ cao nhất có thể

Khi user space đọc:

`bpf_map_lookup_elem()` → `[val_cpu0, val_cpu1, ...]`
`user space sum()` → `total`

Ứng dụng trong Cilium:

- Đếm bytes/packets forwarded per policy
- Đếm packets dropped per reason
- Không bao giờ slow down high-traffic path

Lab Time: Inspect BPF Maps trực tiếp

Chúng ta sẽ thực hành:

1. **List BPF maps:** `bpftool map list` để thấy tất cả maps Cilium tạo.
2. **Xem conntrack table:** `cilium bpf ct list global` — active connections.
3. **Xem policy map:** `cilium bpf policy list` — rules đang enforce.
4. **Demo O(1) vs O(n):** So sánh lookup time iptables vs BPF hash.

👉 Hãy làm theo các bước chi tiết trong file `lab-guide.md`

Tập tiếp theo (Tập 25): Kiến trúc Cilium — Operator, Agent, GoBGP, Hubble so sánh với Calico.